Task 1 — Precision, Recall and F1-Score for Viola-Jones

```
import cv2
import numpy as np
from sklearn.metrics import precision_score, recall_score, f1_score
from google.colab import files

# Upload image
uploaded = files.upload()

# Use sample.jpg
img = cv2.imread("sample.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Viola-Jones face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

faces = face_cascade.detectMultiScale(gray, 1.1, 5)

print("Detected Faces:", len(faces))

# Example ground truth
# Format: x, y, width, height
ground_truth = [(50, 50, 200, 200)]

def iou(box1, box2):
    x1, y1, w1, h1 = box1
    x2, y2, w2, h2 = box2

    xa = max(x1, x2)
    ya = max(y1, y2)
    xb = min(x1+w1, x2+w2)
    yb = min(y1+h1, y2+h2)

    inter = max(0, xb-xa) * max(0, yb-ya)
    area1 = w1*h1
    area2 = w2*h2

    return inter / (area1 + area2 - inter + 1e-6)
```

```
TP = 0

for pred in faces:
    for gt in ground_truth:
        if iou(pred, gt) >= 0.5:
            TP += 1
            break

FP = max(0, len(faces) - TP)
FN = max(0, len(ground_truth) - TP)

precision = TP / (TP + FP + 1e-6)
recall = TP / (TP + FN + 1e-6)
f1 = 2 * precision * recall / (precision + recall + 1e-6)

print("Precision:", round(precision, 3))
print("Recall:", round(recall, 3))
print("F1-Score:", round(f1, 3))
```

sample.jpg(image/jpeg) - 60117 bytes, last modified: 7/30/2026 - 100% done
Saving sample.jpg to sample (1).jpg
Detected Faces: 0
Precision: 0.0
Recall: 0.0
F1-Score: 0.0

Task 2 — YOLO Confidence Scores and NMS

```
!pip install -q ultralytics

from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt

model = YOLO("yolov8n.pt")

img = cv2.imread("sample.jpg")

# Different confidence thresholds
for conf in [0.25, 0.50, 0.75]:

    results = model.predict(
```

```
        source=img,
        conf=conf,
        iou=0.5,
        verbose=False
    )

    result_img = results[0].plot()

    plt.figure(figsize=(8,5))
    plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB))
    plt.title("YOLO Confidence = " + str(conf))
    plt.axis("off")
    plt.show()

    print("Confidence:", conf)
    print("Objects detected:", len(results[0].boxes))
```

45.6/45.6 kB 3.0 MB/s eta 0:00:00

1.4/1.4 MB 37.1 MB/s eta 0:00:00

53.2/53.2 kB 4.0 MB/s eta 0:00:00

64.4/64.4 kB 5.8 MB/s eta 0:00:00
Creating new Ultralytics Settings v0.0.7 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'.
For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% ———————————— 6.2MB 105.6MB/s 0.1s

Confidence: 0.25
Objects detected: 2

Task 3 — Viola-Jones vs Deep Learning Detector

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

img = cv2.imread("sample.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Viola-Jones
haar = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

faces = haar.detectMultiScale(gray, 1.1, 5)

print("Viola-Jones Faces:", len(faces))
```

```
# YOLO
model = YOLO("yolov8n.pt")

results = model.predict(
    source=img,
    conf=0.5,
    verbose=False
)

print("YOLO Objects:", len(results[0].boxes))

# Display Viola-Jones
vj_img = img.copy()

for x, y, w, h in faces:
    cv2.rectangle(
        vj_img,
        (x,y),
        (x+w,y+h),
        (0,255,0),
        2
    )

plt.figure(figsize=(8,5))
plt.imshow(cv2.cvtColor(vj_img, cv2.COLOR_BGR2RGB))
plt.title("Viola-Jones Detection")
plt.axis("off")
plt.show()

# YOLO result
plt.figure(figsize=(8,5))
plt.imshow(
    cv2.cvtColor(results[0].plot(), cv2.COLOR_BGR2RGB)
)
plt.title("YOLO Detection")
plt.axis("off")
plt.show()
```

Confidence: 0.5
Objects detected: 1

Confidence: 0.75
Objects detected: 1

Task 4 — Intersection over Union (IoU)

```
def calculate_iou(box1, box2):

    x1, y1, x2, y2 = box1
    a1, b1, a2, b2 = box2

    xi1 = max(x1, a1)
    yi1 = max(y1, b1)
    xi2 = min(x2, a2)
    yi2 = min(y2, b2)

    intersection = max(0, xi2-xi1) * max(0, yi2-yi1)

    area1 = (x2-x1) * (y2-y1)
    area2 = (a2-a1) * (b2-b1)

    union = area1 + area2 - intersection
```

```
    return intersection / union

ground_truth = (100, 100, 300, 300)
prediction = (120, 120, 310, 310)

iou = calculate_iou(ground_truth, prediction)

print("Ground Truth Box:", ground_truth)
print("Predicted Box:", prediction)
print("IoU:", round(iou, 4))
```

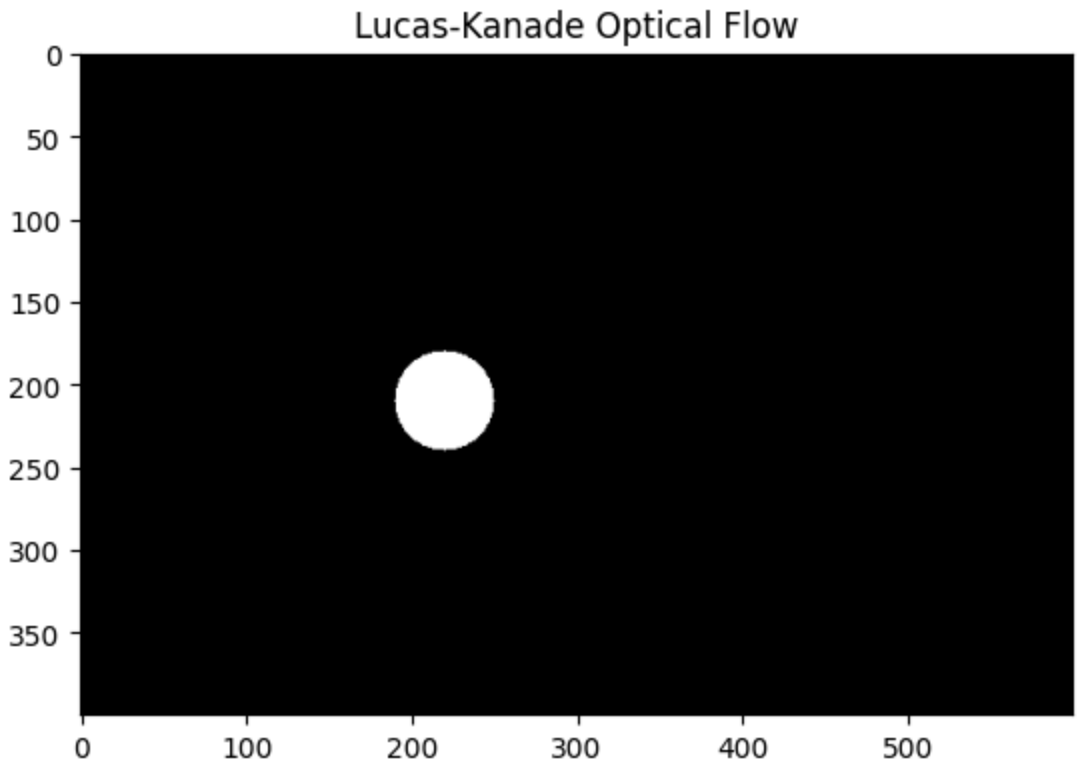
Ground Truth Box: (100, 100, 300, 300)
Predicted Box: (120, 120, 310, 310)
IoU: 0.7414

Task 5 — Lucas-Kanade Optical Flow

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create two frames
frame1 = np.zeros((400,600), dtype=np.uint8)
frame2 = np.zeros((400,600), dtype=np.uint8)

cv2.circle(frame1, (200,200), 30, 255, -1)
cv2.circle(frame2, (220,210), 30, 255, -1)

# Detect corners
features = cv2.goodFeaturesToTrack(
    frame1,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

# Lucas-Kanade
next_points, status, error = cv2.calcOpticalFlowPyrLK(
    frame1,
    frame2,
```

```
    features,
    None
)

good_old = features[status == 1]
good_new = next_points[status == 1]

print("Tracked Points:", len(good_new))

for old, new in zip(good_old, good_new):

    x1, y1 = old.ravel()
    x2, y2 = new.ravel()

    print(
        "Motion:",
        round(x2-x1,2),
        round(y2-y1,2)
    )

plt.imshow(frame2, cmap="gray")
plt.title("Lucas-Kanade Optical Flow")
plt.show()
```

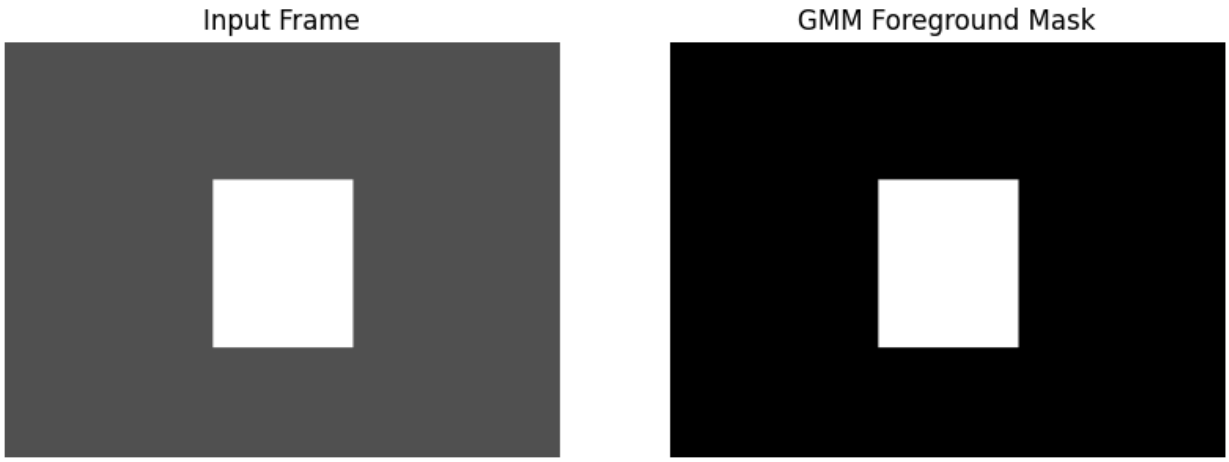
Tracked Points: 12
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0
Motion: 20.0 10.0



Task 6 — Gaussian Mixture Model Background-Foreground Segmentation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create background
background = np.zeros((300,400,3), dtype=np.uint8)
background[:] = (80,80,80)

# Create foreground
frame = background.copy()
cv2.rectangle(frame, (150,100), (250,220), (255,255,255), -1)

# GMM background subtractor
gmm = cv2.createBackgroundSubtractorMOG2(
    history=100,
    varThreshold=16,
    detectShadows=True
```

```
)

# Train background
for i in range(5):
    gmm.apply(background)

# Detect foreground
mask = gmm.apply(frame)

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.title("Input Frame")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(mask, cmap="gray")
plt.title("GMM Foreground Mask")
plt.axis("off")

plt.show()
```



Task 7 — GMM Background Subtraction Accuracy

```
import cv2
import numpy as np

background = np.zeros((300,400,3), dtype=np.uint8)
```

```
background[:] = (50,50,50)

gmm = cv2.createBackgroundSubtractorMOG2(
    history=100,
    varThreshold=16
)

accuracies = []

for i in range(10):

    frame = background.copy()

    x = 50 + i*20

    cv2.rectangle(
        frame,
        (x,100),
        (x+60,200),
        (255,255,255),
        -1
    )

    # Ground truth
    ground_truth = np.zeros((300,400), dtype=np.uint8)

    cv2.rectangle(
        ground_truth,
        (x,100),
        (x+60,200),
        255,
        -1
    )

    mask = gmm.apply(frame)

    # Binary mask
    mask = np.where(mask > 200, 255, 0).astype(np.uint8)

    correct = np.sum(mask == ground_truth)
```

```
    total = ground_truth.size

    accuracy = correct / total

    accuracies.append(accuracy)

print("Frame Accuracies:")

for i, acc in enumerate(accuracies):
    print("Frame", i+1, ":", round(acc,4))

print("Average Accuracy:",
      round(np.mean(accuracies),4))
```

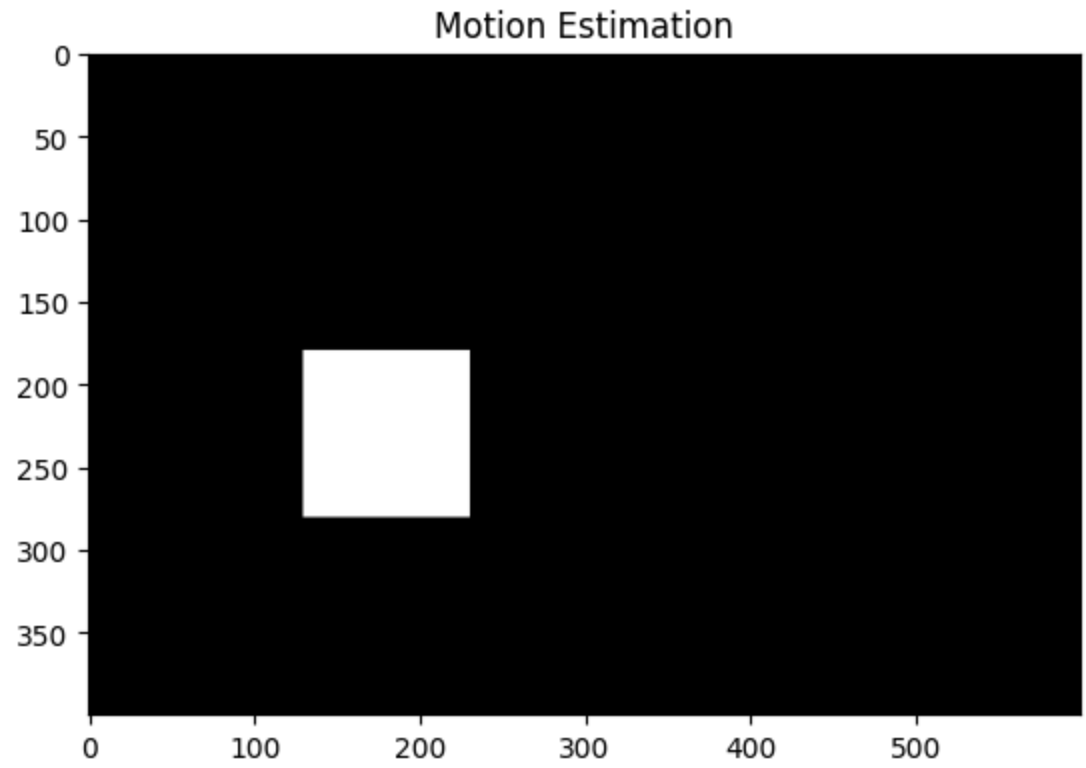
Frame Accuracies:
Frame 1 : 0.9487
Frame 2 : 0.9487
Frame 3 : 0.9487
Frame 4 : 0.9487
Frame 5 : 0.9487
Frame 6 : 0.9655
Frame 7 : 0.9655
Frame 8 : 0.9655
Frame 9 : 0.9655
Frame 10 : 0.9655
Average Accuracy: 0.9571

Task 8 — Motion Estimation Between Two Frames

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

frame1 = np.zeros((400,600), dtype=np.uint8)
frame2 = np.zeros((400,600), dtype=np.uint8)

cv2.rectangle(
    frame1,
    (100,150),
    (200,250),
    255,
    -1
)
```

```
cv2.rectangle(
    frame2,
    (130,180),
    (230,280),
    255,
    -1
)

# Dense optical flow
flow = cv2.calcOpticalFlowFarneback(
    frame1,
    frame2,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

# Average motion
dx = np.mean(flow[:,:,0])
dy = np.mean(flow[:,:,1])

print("Average X Motion:", round(dx,3))
print("Average Y Motion:", round(dy,3))

# Visualization
plt.imshow(frame2, cmap="gray")
plt.title("Motion Estimation")
plt.show()
```

Average X Motion: 0.769
Average Y Motion: 0.984



Task 9 — Reprojection Error in Structure-from-Motion

```
import numpy as np
import cv2

# 3D points
object_points = np.array([
    [0,0,0],
    [1,0,0],
    [1,1,0],
    [0,1,0],
    [0,0,1],
    [1,0,1]
], dtype=np.float32)

# Camera matrix
K = np.array([
    [800,0,320],
    [0,800,240],
    [0,0,1]
], dtype=np.float32)
```

```
# Camera pose
rvec = np.zeros((3,1), dtype=np.float32)
tvec = np.array([[0],[0],[5]], dtype=np.float32)

# Project 3D points
image_points, _ = cv2.projectPoints(
    object_points,
    rvec,
    tvec,
    K,
    None
)

image_points = image_points.reshape(-1,2)

# Add small measurement noise
observed_points = image_points + np.random.normal(
    0, 2, image_points.shape
)

# Reprojection error
errors = np.linalg.norm(
    image_points - observed_points,
    axis=1
)

print("Point-wise Reprojection Errors:")
print(errors)

print(
    "Mean Reprojection Error:",
    round(np.mean(errors),4),
    "pixels"
)
```

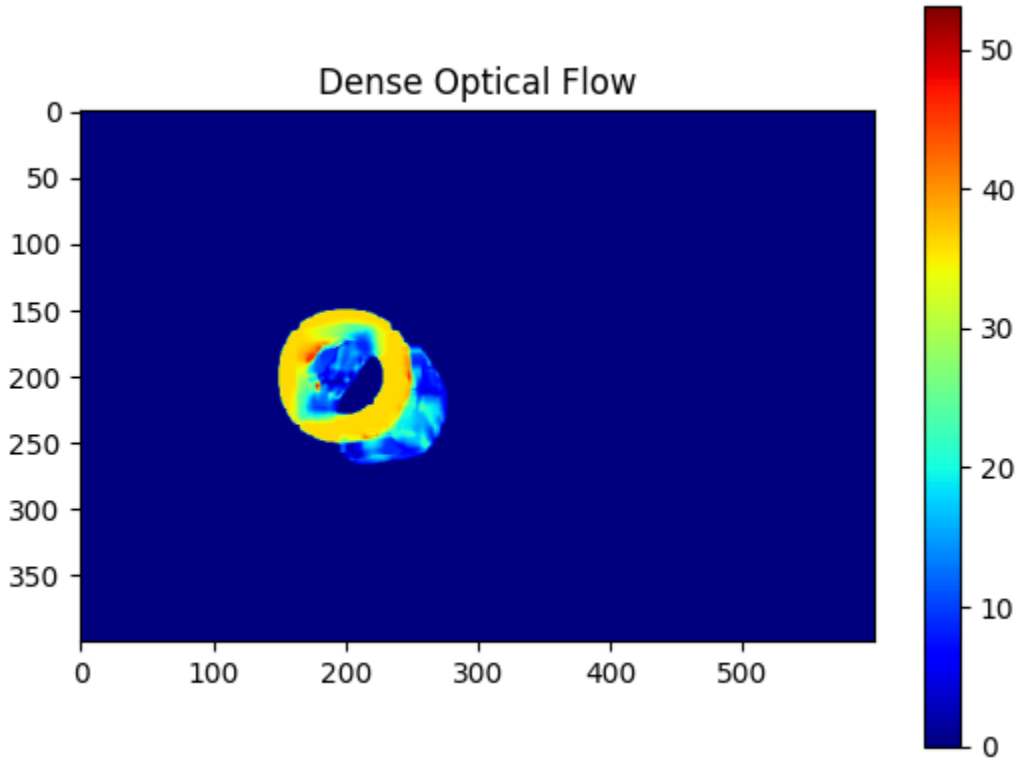
Point-wise Reprojection Errors:
[3.3616 2.2079 5.1814 3.7997 3.1209 3.7478]
Mean Reprojection Error: 3.5699 pixels

Task 10 — Dense vs Sparse Optical Flow

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

frame1 = np.zeros((400,600), dtype=np.uint8)
frame2 = np.zeros((400,600), dtype=np.uint8)

cv2.circle(frame1, (200,200), 40, 255, -1)
cv2.circle(frame2, (230,220), 40, 255, -1)

# Sparse Optical Flow
points = cv2.goodFeaturesToTrack(
    frame1,
    maxCorners=50,
    qualityLevel=0.01,
    minDistance=5
)

next_points, status, error = cv2.calcOpticalFlowPyrLK(
    frame1,
    frame2,
    points,
    None
)

good_old = points[status==1]
good_new = next_points[status==1]

print("Sparse Flow Points:", len(good_new))

# Dense Optical Flow
dense_flow = cv2.calcOpticalFlowFarneback(
    frame1,
    frame2,
    None,
    0.5, 3, 15, 3, 5, 1.2, 0
)

magnitude = np.sqrt(
    dense_flow[:,:,0]**2 +
```

```
    dense_flow[:,:,1]**2
)

print(
    "Average Dense Flow Magnitude:",
    round(np.mean(magnitude),4)
)

plt.imshow(magnitude, cmap="jet")
plt.title("Dense Optical Flow")
plt.colorbar()
plt.show()
```

Sparse Flow Points: 28
Average Dense Flow Magnitude: 1.0435



Task 11 — Detection Speed vs Accuracy: Viola-Jones vs YOLO

```
import cv2
import time
from ultralytics import YOLO
```

```python
img = cv2.imread("sample.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Viola-Jones
haar = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)

start = time.time()

faces = haar.detectMultiScale(
    gray,
    1.1,
    5
)

vj_time = time.time() - start

# YOLO
model = YOLO("yolov8n.pt")

start = time.time()

results = model.predict(
    img,
    conf=0.5,
    verbose=False
)

yolo_time = time.time() - start

print("----- PERFORMANCE COMPARISON -----")

print(
    "Viola-Jones Time:",
    round(vj_time,4),
    "seconds"
)
```

```
print(
    "Viola-Jones Detections:",
    len(faces)
)

print(
    "YOLO Time:",
    round(yolo_time,4),
    "seconds"
)

print(
    "YOLO Detections:",
    len(results[0].boxes)
)
```

----- PERFORMANCE COMPARISON -----
Viola-Jones Time: 0.2918 seconds
Viola-Jones Detections: 0
YOLO Time: 0.2362 seconds
YOLO Detections: 1

Task 12 — Confusion Matrix for Multi-Class Object Recognition

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    accuracy_score,
    precision_score,
    recall_score
)

# Actual classes
y_true = [
    "Car","Car","Car",
    "Bus","Bus","Bus",
    "Bike","Bike","Bike"
]
```

```
# Predicted classes
y_pred = [
    "Car","Car","Bus",
    "Bus","Bus","Car",
    "Bike","Car","Bike"
]

classes = ["Car","Bus","Bike"]

# Confusion Matrix
cm = confusion_matrix(
    y_true,
    y_pred,
    labels=classes
)

print("Confusion Matrix:")
print(cm)

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

precision = precision_score(
    y_true,
    y_pred,
    average="macro"
)

recall = recall_score(
    y_true,
    y_pred,
    average="macrao"
)

print("Accuracy:", round(accuracy,3))
print("Precision:", round(precision,3))
print("Recall:", round(recall,3))

# Display
```

```
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=classes
)

disp.plot()
plt.title("Multi-Class Object Recognition")
plt.show()
```

Confusion Matrix:
[[2 1 0]
 [1 2 0]
 [1 0 2]]
Accuracy: 0.667
Precision: 0.722
Recall: 0.667